



Contact Session 5

Local Search and Optimization Problems

Artificial and Computational Intelligence — DSE ZG557

Focus: local search, hill climbing, simulated annealing and local beam search



BITS Pilani

Pilani Campus

Akhil P V | BITS Pilani WILP

Reference: T1, Chapter 4.1

From CS4 to CS5



CS3–CS4

Find a path from a start state to a goal.

- Route finding, A*, shortest path.
- The action sequence is the answer.

CS5

Improve a complete candidate solution.

- N-Queens, scheduling, layout design, TSP.
- The final configuration is the answer.

PATH SEARCH



LOCAL SEARCH



Key distinction

Path search returns a sequence of actions; local search returns a state.

Learning Objectives



1. Explain local search and optimization problems.
2. Distinguish path search from local search.
3. Explain state-space landscapes.
4. Trace hill climbing.
5. Explain local maxima, ridges and plateaux.

6. Explain random restart and sideways moves.
7. Explain simulated annealing.
8. Explain local beam search.
9. Compare the three methods.

Session focus

Objectives 1–3 establish what local search is; objectives 4–6 concern hill climbing and its failure modes; objectives 7–9 cover the remedies and the comparison.

Session Roadmap



1 Why local search?	2 Complete-state formulation	3 State-space landscape
4 Hill climbing	5 Problems with hill climbing	6 Random restart and sideways moves
7 Simulated annealing	8 Local beam search	9 Comparison and exam points

Note

Genetic algorithms belong to CS6; they appear here only as a closing bridge.

Why Local Search?

In many problems, the path to the solution is not important.

Only the final state matters.

Problem	What matters?
N-Queens	Final non-attacking arrangement
Timetable scheduling	Valid and efficient timetable
Factory layout	Low-cost arrangement
Travelling Salesman Problem	Short tour
Portfolio optimization	Good allocation

Key takeaway

Local search is suitable when final solution quality matters more than the action sequence used to reach it.

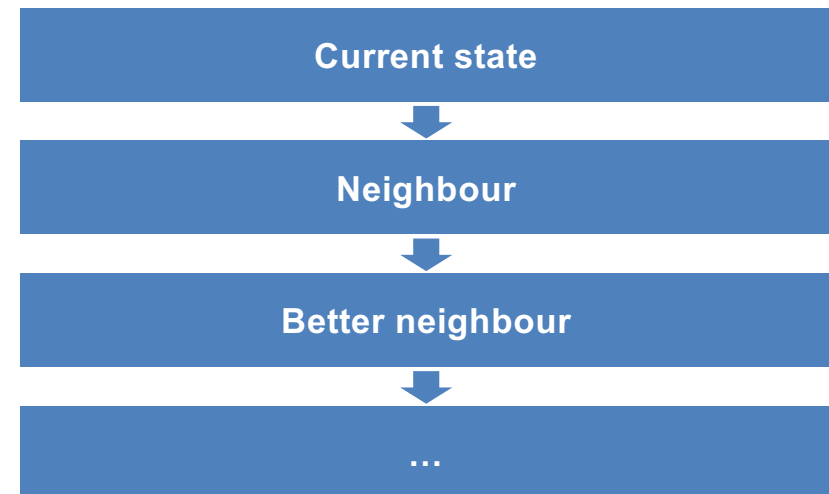
Local Search: Definition



Local search algorithms

- keep one or a few current states;
- move to neighbouring states;
- usually do not store the full path;
- use little memory;
- are useful for very large or continuous spaces.

Mental model



Key takeaway

Local search trades systematic path exploration for low memory use and direct improvement of the solution.

Complete-State Formulation



Q							
						Q	
				Q			
							Q
	Q						
			Q				
					Q		
		Q					

8-Queens: one queen per column

State

8 queens already placed, one per column

Move

Move one queen within its own column

Evaluation

$h(n)$ = number of attacking queen pairs

Goal

$h(n) = 0$

Key contrast

We do not build a solution piece by piece; we start from a full but imperfect arrangement and repair it.



Objective and Evaluation Functions

MAXIMIZATION

Higher is better

Find the state with the maximum value.

Example: maximise profit.

MINIMIZATION

Lower is better

Find the state with the minimum cost.

Examples: conflicts, distance, delay.

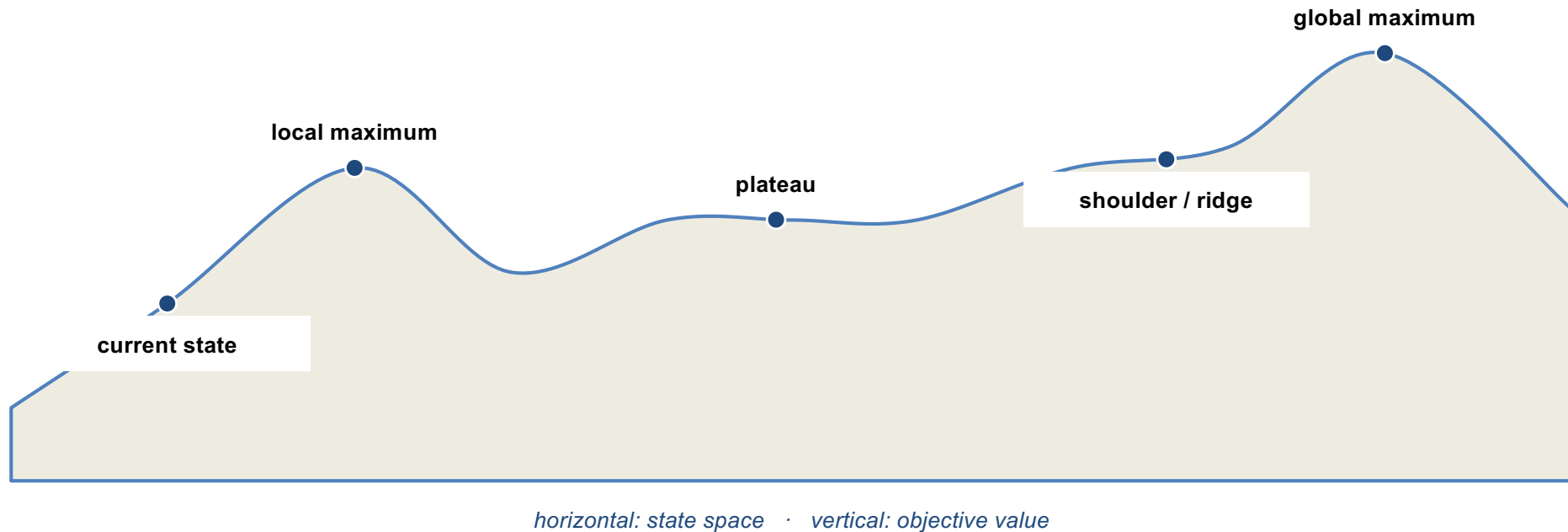
For 8-Queens: $h(n)$ = number of attacking queen pairs Goal: $h(n) = 0$

A minimization problem — a lower conflict count is a better state.

Key takeaway

Local search requires a numerical measure of improvement; without one, “better neighbour” has no meaning.

State-Space Landscape



Key message

Local search moves from one nearby point to another; it never sees the whole landscape at once.

Local Search vs Classical Search

Feature	Classical search	Local search
Main goal	Find a path to a goal	Improve the current state
Stores path?	Usually yes	Usually no
Memory	Can be large	Usually small
State	Partial / path-based	Complete candidate solution
Example	A* route search	N-Queens optimization

Key takeaway

Local search is a different problem-solving style, not just another graph-search algorithm. Compare it on formulation, not only on speed.

Hill Climbing: Basic Idea



At each step

1. Examine the neighbours of the current state.
2. Select a better neighbour.
3. Move to it.
4. Stop when no neighbour is better.

Mental model

Move in the direction of improvement

No memory of where it has been.

No plan for where it is going.

Key takeaway

Hill climbing repeatedly improves the current state using local information only, and halts at the first state with no better neighbour.



Hill Climbing: Algorithm

```
HILL-CLIMBING(problem)
```

```
  current ← initial state
```

```
  loop:
```

```
    neighbour ← best successor of current
```

```
    if VALUE(neighbour) ≤ VALUE(current):  
      return current
```

```
  current ← neighbour
```

For minimization

- Choose the successor with the LOWEST cost.
- Stop when no successor has a lower cost.
- Test becomes: if $\text{COST}(\text{neighbour}) \geq \text{COST}(\text{current})$, return current.

Important

The textbook form is steepest-ascent maximization. For N-Queens we typically minimise conflicts, so the comparison reverses. State which convention you are using.

Hill Climbing on 8-Queens

		Q					
				Q			
						Q	
Q							
							Q
	Q						
			Q				
					Q		

current board, $h = 17$

Current
 $h = 17$



Best neighbour
 $h = 12$

Move, because $12 < 17$ — the successor lowers the conflict count.

Sample successor (move one queen in its column)	h
Queen in column 3 → row 2	12
Queen in column 1 → row 6	14
Queen in column 6 → row 5	15
Queen in column 8 → row 1	16

Only a few of the 56 successors are shown.

Key takeaway

Hill climbing chooses the locally best improvement, and continues until $h = 0$ or no neighbour improves the value.

Hill Climbing Variants

Variant	Idea
Steepest-ascent	Choose the best improving neighbour
Stochastic hill climbing	Choose randomly among the improving moves
First-choice hill climbing	Generate random successors until one improves
Random-restart hill climbing	Repeatedly begin from new random states

Why the variants exist

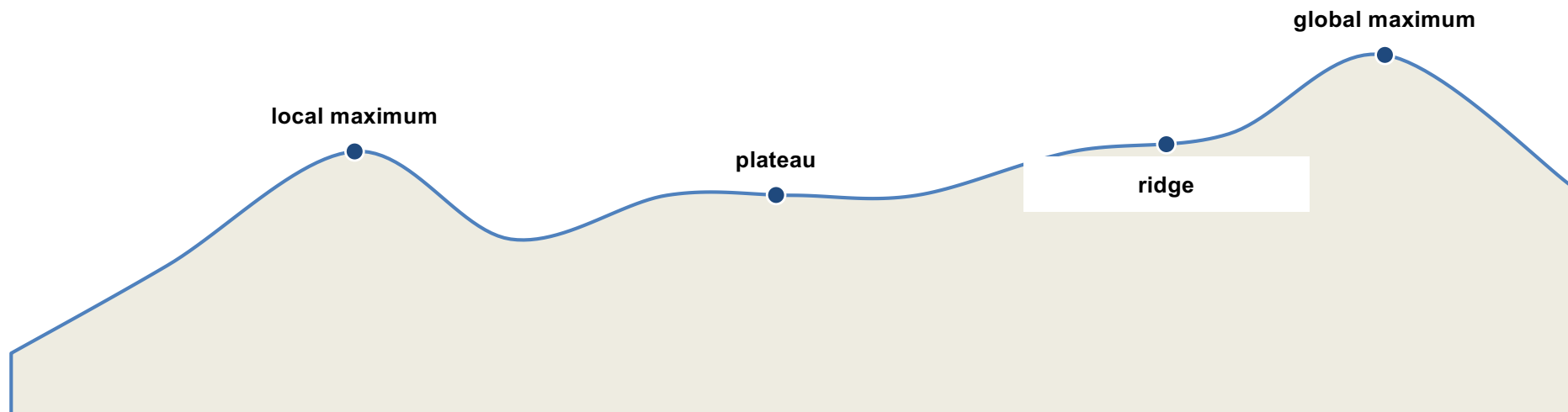
They address three different difficulties: costly neighbour generation, a poor starting state, and local traps.

Why Hill Climbing Can Fail

1 Local maximum / minimum

2 Ridge

3 Plateau



Main problem

Local improvement does not guarantee a global optimum.



Local Maximum / Local Minimum

For maximization

A local maximum is better than all neighbouring states, but worse than the global maximum.

For minimization

A local minimum has lower cost than all its neighbours, but is not the global minimum.

Example — 8-Queens

A state with $h = 1$ may still be stuck if every single-queen move increases the conflict count.

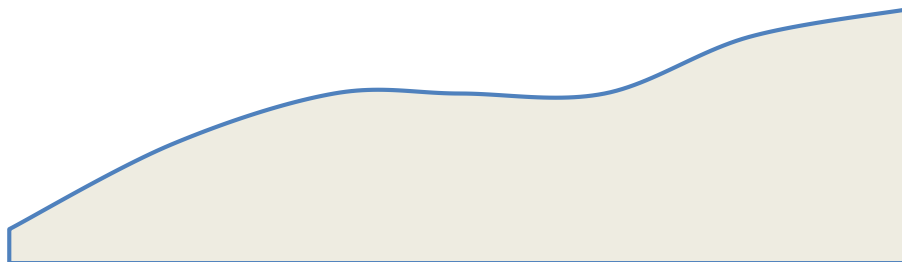
Key takeaway

Locally best does not necessarily mean globally best.

Plateaux and Ridges



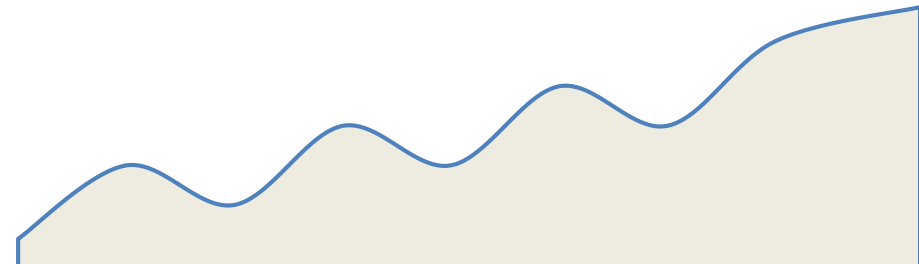
PLATEAU



A flat region where neighbouring states have equal value.

Problem: No clear direction of improvement.

RIDGE



A narrow improvement path that may require sideways or indirect movement.

Problem: Available moves do not align with the direction of improvement.

Exam point

A plateau is flat; a ridge is not flat but is narrow. They fail for different reasons.

Sideways Moves



$$\text{VALUE}(\text{neighbour}) = \text{VALUE}(\text{current})$$

A sideways move goes to a neighbour with the same value.

Useful when

- the algorithm is on a plateau;
- progress may become possible later.

Risk

- infinite wandering on a flat region;
- no guarantee the plateau is a shoulder.

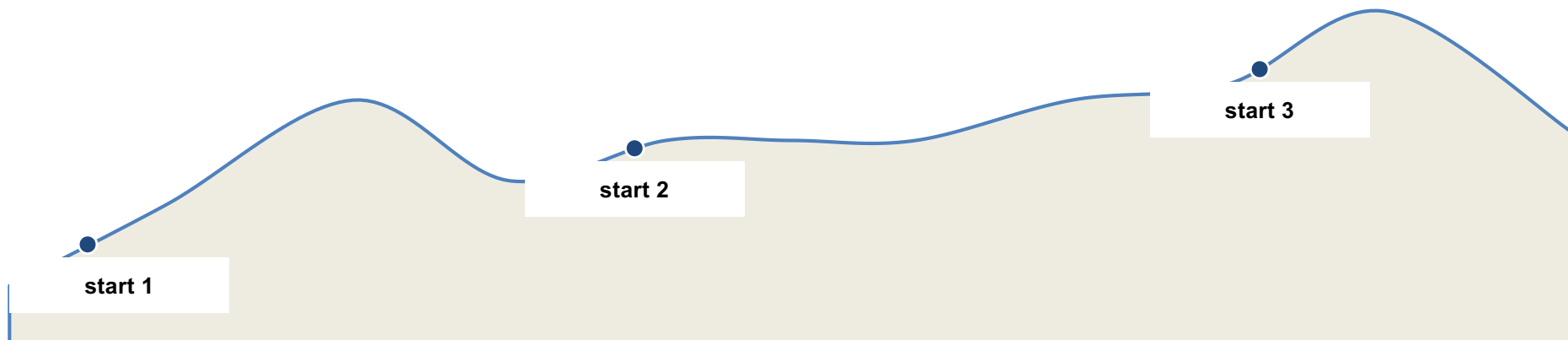
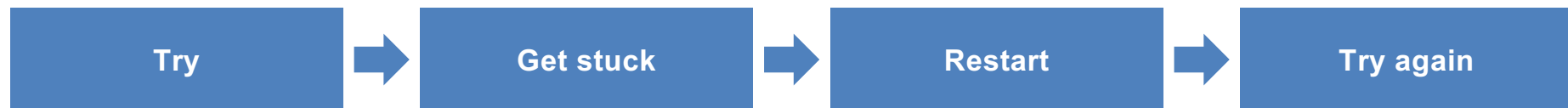
Practical solution

- limit the number of consecutive sideways moves;
- restart the count after any real improvement.

Key takeaway

Sideways moves can help cross plateaux, but they need an explicit limit.

Random-Restart Hill Climbing



Different starting states reach different regions of the landscape. Stop when a solution is found, or when the restart limit is reached.

Key takeaway

Random restarts reduce dependence on the initial state.

From Hill Climbing to Simulated Annealing



Hill climbing

Always rejects worse moves.

Problem: gets trapped at local optima.

→ only uphill / better moves



Simulated annealing

Sometimes accepts worse moves.

Purpose: escape local optima early, settle later.

→ mostly better, with occasional worse moves

Key takeaway

Simulated annealing balances local improvement with controlled randomness — the amount of randomness is reduced as the search proceeds.

The Annealing Analogy

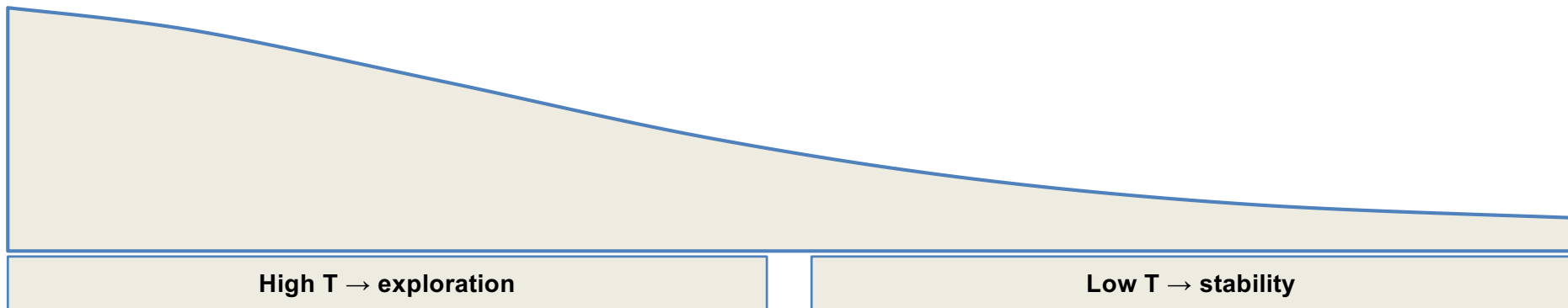


PHYSICAL ANNEALING

1. Heat the material.
2. Atoms move freely.
3. Cool it slowly.
4. It settles into a low-energy structure.

SIMULATED ANNEALING

1. Start with a high temperature.
2. Explore freely.
3. Reduce the temperature gradually.
4. Settle into a good solution.



Key takeaway

Temperature controls the willingness to explore worse states.

Simulated Annealing: The Acceptance Rule



At each step

1. Pick a random successor.
2. If it improves the state, accept it.
3. If it worsens the state, accept it only with a probability P .

$$P = e^{(\Delta E / T)}$$

applies only when the move is worse

$\Delta E < 0$ for a worse move (maximization).

T = temperature.

As T decreases, worse moves become less likely to be accepted.

Key takeaway

A large worsening is less likely to be accepted than a small one, and a high temperature makes any worse move easier to accept.



Simulated Annealing: Algorithm

```
SIMULATED-ANNEALING(problem, schedule)
```

```
  current ← initial state
```

```
  for t = 1 to ∞:
```

```
    T ← schedule(t)
```

```
    if T = 0:
```

```
      return current
```

```
    next ← randomly selected successor
```

```
     $\Delta E \leftarrow \text{VALUE}(\text{next}) - \text{VALUE}(\text{current})$ 
```

```
    if  $\Delta E > 0$ :
```

```
      current ← next
```

```
    else:
```

```
      accept next with probability  $e^{-(\Delta E/T)}$ 
```

Reading the algorithm

- This is the maximization form.
- If next is better, accept it.
- If next is worse, possibly accept it — depending on the degree of worsening and on T.
- The schedule decides how quickly T reaches zero.

Key takeaway

Simulated annealing is stochastic local search with controlled downhill moves.

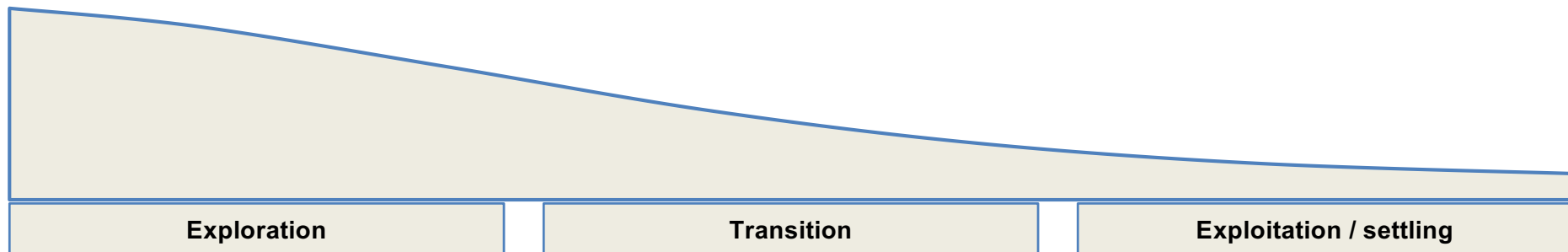
The Cooling Schedule

High T

- More exploration.
- Worse moves accepted more often.

Low T

- Less exploration.
- Behaves more like hill climbing.



Cooling too fast may leave the search stuck; cooling too slowly may simply take too long.

Key takeaway

The cooling schedule strongly influences the exploration–exploitation balance.



When Is Simulated Annealing Useful?

Useful when

- the search space is very large;
- many local optima exist;
- occasional worse moves may escape traps;
- approximate good solutions are acceptable.

Typical applications

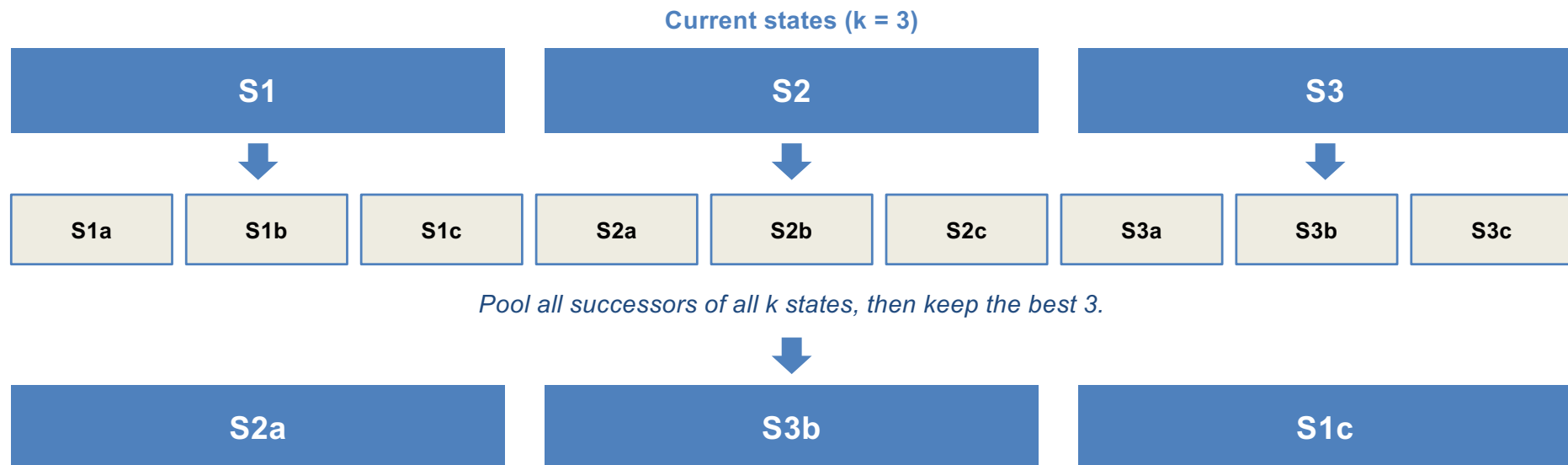
- VLSI layout;
- factory scheduling;
- routing;
- resource allocation;
- timetable generation.

Key takeaway

Simulated annealing is useful on rugged optimization landscapes, where good solutions are acceptable and proof of optimality is not required.

Local Beam Search: Basic Idea

Hill climbing keeps 1 current state. Local beam search keeps k current states.



Key takeaway

Local beam search explores several regions of the landscape simultaneously.



Local Beam Search: Algorithm

```
LOCAL-BEAM-SEARCH(problem, k)
```

```
  current_states ← k random states
```

```
  loop:
```

```
    generate all successors  
      of current_states
```

```
    if any successor is a goal:  
      return solution
```

```
    current_states ← best k successors
```

Important

This is NOT the same as running k independent hill climbers.

Why? The next k states are selected from the combined successor pool.

A state that produces poor successors simply disappears.

Key takeaway

Local beam search shares information across the population of current states: useful regions attract more of the search effort.



Local Beam Search vs Random Restart

Feature	Random-restart hill climbing	Local beam search
Number of states	One run at a time	k states together
Search behaviour	Independent starts	Information shared
Resource movement	Restart after failure	Concentrate on the best regions
Risk	Repeated poor starts	Loss of diversity

Key takeaway

Beam search shares information between its states, but for that same reason it may collapse into a single region of the landscape.

Stochastic Beam Search



Problem with ordinary local beam search: the k states may become too similar.

Ordinary beam search

Always keeps the best k successors.
The population can collapse into one region.

Stochastic beam search

Selects successors probabilistically.
Better states get a higher probability; weaker states may still survive.

state A — best

state B

state C

state D

Segment width = selection probability. A weaker state keeps a small but non-zero chance of survival.

Benefit

Probabilistic selection maintains diversity in the set of current states.

Comparing the CS5 Algorithms

Algorithm	Memory	Escapes local optima?	Main idea
Hill climbing	Very low	Usually no	Move to a better neighbour
Random restart	Low	Better chance	Try multiple starting points
Simulated annealing	Very low	Yes, probabilistically	Allow some worse moves early
Local beam search	Depends on k	Partly	Keep k states and pool successors

How to use this slide

The methods differ mainly in how they deal with local traps and how much exploration they buy. No method is universally best.

Activity: Choose the Method



Which method would you choose?

1. N-Queens with many local minima.
2. Factory scheduling with complex constraints.
3. A huge search space with limited memory.
4. An optimization problem where diversity matters.
5. A problem where the path itself matters.

Possible directions

- random-restart hill climbing;
- simulated annealing;
- local beam search;
- classical search such as A*.

Discussion, not a quiz

Not every scenario has a single universally correct answer. Ask students to justify the choice from the structure of the problem.

Common Mistakes



Avoid

1. Treating local search as path search.
2. Assuming local search is always complete.
3. Assuming hill climbing always finds the global optimum.
4. Confusing a local optimum with a global optimum.
5. Ignoring plateaux and ridges.
6. Thinking simulated annealing always accepts worse moves.
7. Thinking local beam search is the same as random restart.
8. Forgetting that k affects both memory and diversity.

A simple check

If in doubt, ask three questions: what is the state, what is the objective function, and what counts as a neighbour?

Exam-Focused Summary



LOCAL SEARCH

Improve the current state; usually ignore the path.

HILL CLIMBING

Move to a better neighbour; stop when none is better.

SIMULATED ANNEALING

Sometimes accept worse moves, controlled by temperature.

LOCAL BEAM SEARCH

Keep k states and select the best successors from the pool.

Key landscape problems

local maxima and minima · ridges · plateaux

Key takeaway

CS5 is about optimization through local movement in the state space.

CS5 Exit Check



Answer briefly

1. Why is N-Queens suitable for local search?
2. What is a complete-state formulation?
3. Why can hill climbing get stuck?
4. What is a sideways move?
5. Why does simulated annealing accept worse moves?
6. How does temperature affect simulated annealing?
7. How is local beam search different from random restart?

Teaching note

Use the final five minutes for these, or assign them as a post-class self-check before the CS6 session.

Bridge to CS6: Genetic Algorithms



Local beam search



Stochastic beam search



Genetic algorithm

Connection to CS5

- Local beam search keeps multiple states.
- Stochastic beam search selects probabilistically.
- Both already work with a set of candidates.

Genetic algorithms add

- population;
- fitness;
- selection;
- crossover;
- mutation.

Scope note

This is a conceptual bridge only. Genetic algorithms are taught in CS6.

References



Primary course source

BITS Pilani WILP Digital Learning Handout
Artificial and Computational Intelligence
DSE ZG557

Prescribed textbook

Russell and Norvig
Artificial Intelligence: A Modern Approach
Third edition

Sections used

§4.1 Local Search Algorithms and Optimization Problems · §4.1.1 Hill-Climbing Search
§4.1.2 Simulated Annealing · §4.1.3 Local Beam Search

Study note

Students should revise Chapter 4.1 before the CS6 session.